

Guardian Eye: An AI Safety Sentinel for Blast Zones

Real-time person detection, danger-zone reasoning, and blast-control gating — powered by Gemma 4

Build with Gemma 4 — AI Durg

Track: Open Track

TEAM DETAILS:

- **MAMIDIPALLI SATHWIK**
- **VANGAPATY SREE KRISHNA**
- **REVA THATIKONDA**

The Problem

On mining and blasting sites, the single most dangerous failure mode isn't equipment — it's a person being in the wrong place at the wrong time. A worker who wanders into a blast radius during a countdown, or who is missing required safety gear near a restricted zone, is a fatality risk that a human monitor watching a bank of CCTV feeds can easily miss. Guardian Eye is built to close that gap: a camera-based sentinel that watches restricted zones continuously, reasons about what it sees using Gemma 4, and — most importantly — has the authority to halt a blast countdown the instant it detects a violation.

Critically, "a person in the wrong place" is not a single failure mode — it is two, and each demands a different system response. Guardian Eye is therefore built to operate across two distinct modes:

- **Pre-Blast Verification Mode** — before the countdown is armed, the system must confirm the danger zone is fully clear. If a person is detected during this window, Guardian Eye withholds the blast entirely: the timer is **LOCKED** and cannot be started until the area is re-verified as clear.
- **Active Blast Countdown Mode** — once a countdown is armed and running, the same detection must trigger a different, faster response: an immediate **ABORT** of the live countdown. A person entering the zone mid-countdown is a more urgent event than one detected pre-arm, and the system's action reflects that.

Treating these as one undifferentiated "violation detected" state would blur an important distinction: preventing a blast from starting is a lower-urgency safeguard than aborting one already in progress. Guardian Eye's mode-aware response keeps its authority over the blast sequence proportionate to the actual danger at each stage.

Architecture

Guardian Eye is a three-stage pipeline:

- Perception (perception.py) — YOLOv8 runs on each frame of the live camera feed (or an uploaded CCTV clip). For every detected person, we check their bounding box against a configurable danger-zone polygon and crop the person out of the frame for closer inspection.
- Intelligence (brain.py) — Gemma 4 receives the cropped image context along with the person's zone status (restricted vs. safe) and reasons about the situation, grounded by a RAG step over the site's actual Safety Manual PDF. Gemma returns a structured verdict — CRITICAL (person in the blast radius) or COMPLIANCE (a PPE/procedural gap) — with a plain-language reason.
- Action (action_interface.py / main.py) — A Streamlit "Command Center" dashboard consumes Gemma's verdicts in real time. A CRITICAL verdict actively aborts an in-progress blast countdown and locks the start control until the area is confirmed clear. Role-based access (Manager vs. Employee) restricts who can arm the countdown or manually trigger the site buzzer, and a strobing on-screen alarm plus an audio siren covers both visual and hearing-impaired workers.

How Gemma 4 Is Used

- Grounded reasoning via RAG: relevant passages from the actual safety manual PDF are retrieved and injected into the prompt, so verdicts trace back to a real, site-specific policy rather than a generic notion of safety.
- Structured decision output: the prompt constrains Gemma to a strict JSON contract (priority, msg, reason, identity), which lets the dashboard consume its output programmatically and drive a real control-flow decision — aborting the blast — rather than just displaying text.
- Severity classification tied to physical context: the same prompt handles CRITICAL zone intrusions and COMPLIANCE (PPE/procedural) gaps, using the danger-zone boolean from the perception layer as grounding context.

Design Decision: Separating Geometry from Judgment

A tempting shortcut would have been to ask Gemma to look at a full frame and decide everything. We deliberately split this instead: YOLOv8 and polygon math handle the deterministic geometric question (is this bounding box inside the restricted area), and Gemma 4 handles the judgment question (given this situation and our actual safety rules, what should happen). Each component does what it is actually good at — a vision-language model guessing pixel coordinates is a worse detector than a purpose-built detection model, and a hardcoded rule engine is a worse judge of nuanced compliance questions than an LLM grounded in real policy text.

From Detection to Action

Most "AI safety monitoring" demos stop at an alert badge. Guardian Eye's differentiator is that the AI's verdict has real authority over a physical process in the simulation: a CRITICAL signal during an active countdown flips it to an ABORTED state and disables the start control until a human re-clears the area. This closes the loop between "the AI noticed something" and "the dangerous action actually stops" — the property that matters most on a real blast site.

Sprint Challenges

The main challenge was keeping the reasoning loop fast enough to matter in a live-monitoring context while still grounding every verdict in the actual safety manual — retrieving only the top relevant chunks per query kept latency low enough for near-real-time operation.

The second challenge was the handoff between the perception and intelligence work: standardizing on a clear manifest format (frame ID, timestamp, bounding box, danger-zone flag, image path) let the Gemma reasoning layer be built and tested independently of the live camera loop, which mattered given the one-day timeframe.

Safety, Access, and Accountability

- Only Manager-role users can arm the blast countdown or manually trigger the site-wide buzzer; Employee-role logins can view alerts but not control the blast sequence.
- Every CRITICAL verdict is logged with a timestamp and reason and stays visible in the on-screen safety log for the shift.
- The dashboard defaults to CCTV disconnected on every login, so an operator must explicitly grant camera access each session rather than monitoring silently running by default.

Live Demo Access

The Command Center enforces role-based login before any control surface is reachable, so evaluators will need a demo credential to exercise the system end-to-end. Two roles are provisioned for this purpose:

Role	Capabilities
Manager	Connect/disconnect CCTV, trigger buzzer, arm/abort blast countdown, view alerts

Employee	View live feed and redacted safety log; no blast control
----------	--

Screenshots

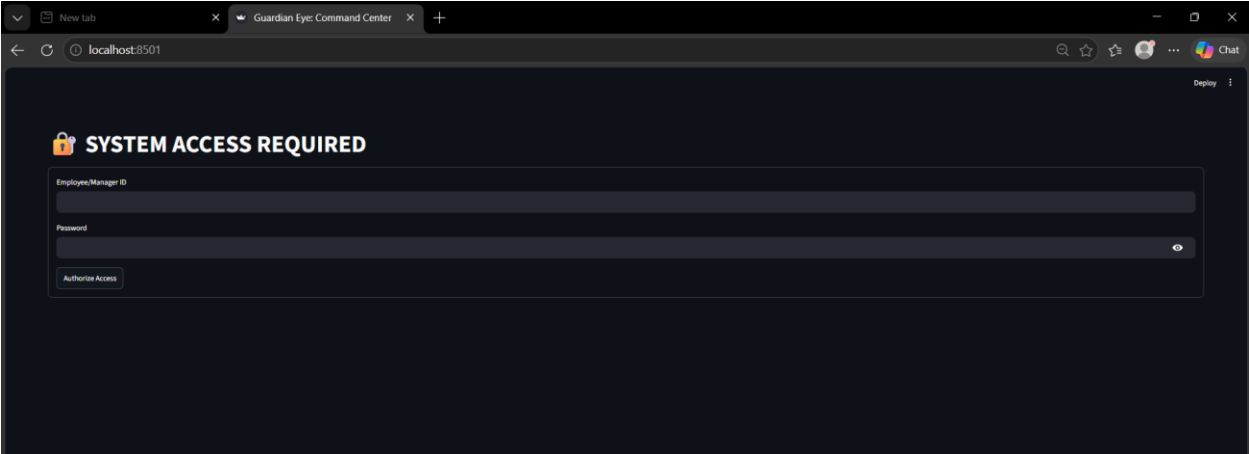


Figure 1 — Role-gated login screen. No dashboard control is reachable without authorization.

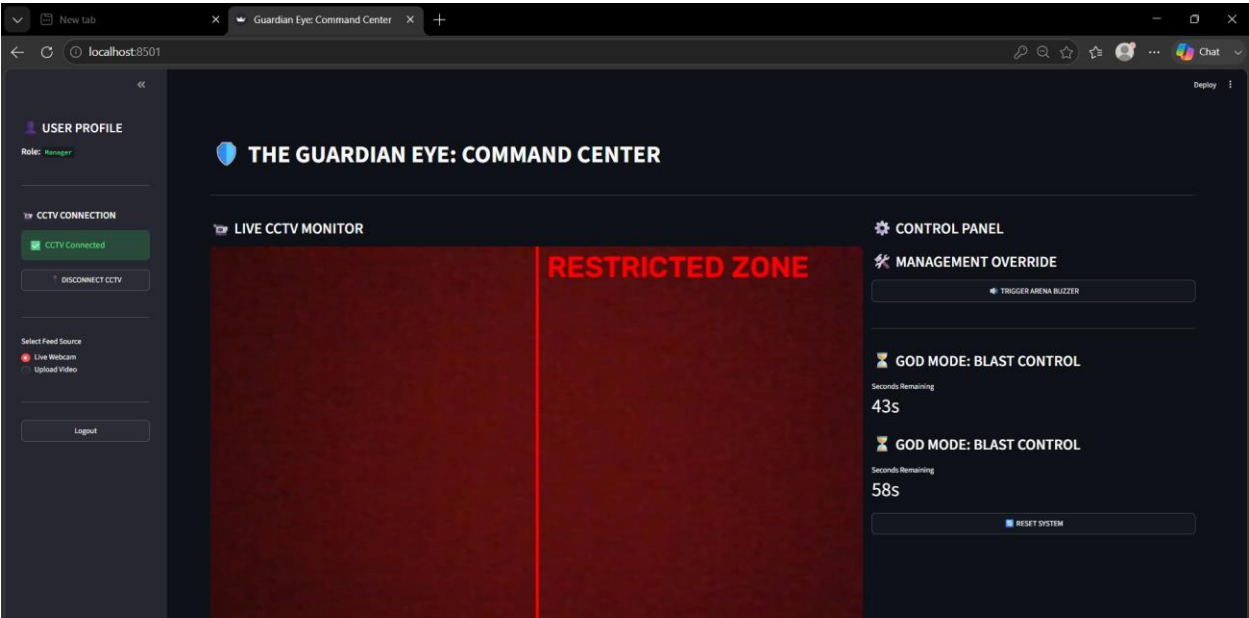


Figure 2 — Manager view: live restricted-zone overlay and active God Mode blast countdown.

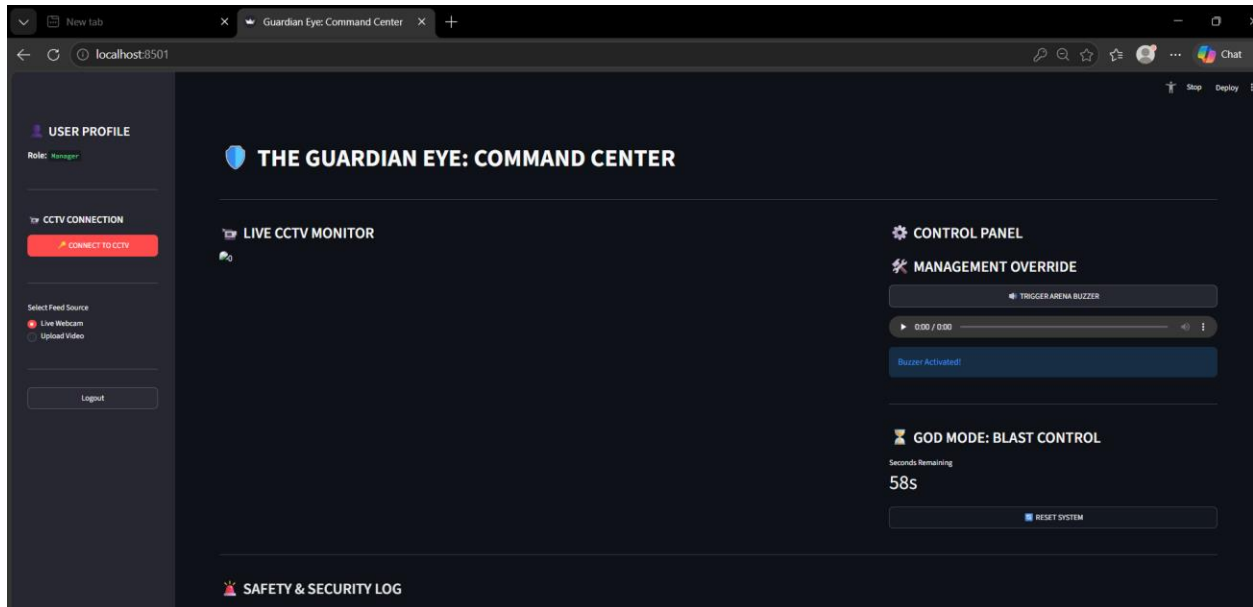


Figure 3 — Manager-only arena buzzer trigger, confirmed "Buzzer Activated!".

Why This Matters

Guardian Eye isn't trying to replace a safety officer. It's trying to guarantee that the one failure mode that kills people on a blast site — a person in the wrong place during a countdown — gets caught and acted on even in the second a human monitor looks away. Grounding Gemma's judgment in the site's real safety manual, and giving that judgment actual authority over the blast sequence, is what makes this more than a detection demo.